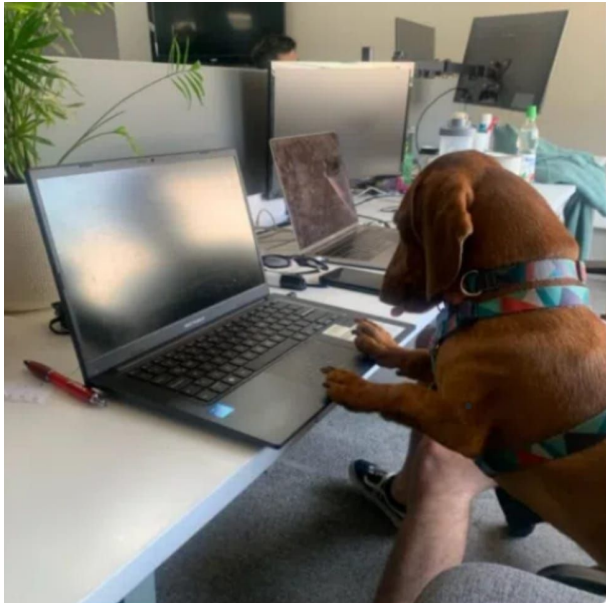
Programación Avanzada
IIC2233 2026-1

Jaime Middleton - Pablo Araneda - Antonio López - Tamara Vidal - Daniela Concha

Anuncios

26 de marzo de 2026



1. Hoy tenemos la Actividad 3.
    - Recuerden firmar la lista y asistir a la sección en la que están inscritos, si quedan ausentes tendrán 0 puntos.
2. Hoy se libera la Tarea 1.
3. Recuerden llenar la ECA todas las semanas para la bonificación extra a fin de semestre ✨.

Herencia en OOP

Herencia en OOP

- ¿Cuál es la diferencia entre atributos de clase y atributos de instancia?
- ¿Qué significa heredar en OOP?
- ¿Cuál es la relación entre una *superclase* y una *subclase*?
- ¿Por qué y cuándo usamos el método **`super()`**?
- ¿Hay algún límite en la herencia en OOP?

Ejemplo de herencia

```python
class Perro:
    def __init__(self, nombre):
        self.nombre = nombre

    def ladrar(self):
        print("Woof woof!")
```

```python
class Dachshund(Perro):
    def __init__(self, nombre, sabe_ladrar):
        super().__init__(nombre)
        self.sabe_ladrar = sabe_ladrar

    def sit(self):
        print("Crees que soy esclavo?!?!")

    def defiende_la_casa(self):
        if self.sabe_ladrar:
            self.ladrar()
            print("Alejate de mi comida!!")
        else:
            print("Extraño me hace cariño?")
```

```python
class Collie(Perro):
    def __init__(self, nombre, sabe_defender):
        super().__init__(nombre)
        self.sabe_defender = sabe_defender

    def fetch(self):
        print("Voy corriendo a buscar pelota!!")

    def defiende_la_casa(self):
        if self.sabe_defender:
            self.ladrar()
            print("Alejate de mi casa!!")
        else:
            print("Iré a avisarle a mi mamá!")
```

Polimorfismo

Polimorfismo

- ¿Qué implica realizar *overriding* en Python?
- *Overloading* no existe en Python.
- ¿Y los operadores aritméticos en Python?

Ejemplo de *overriding* y "*overloading*"

```python
class Perro:
    def __init__(self, nombre):
        self.nombre = nombre

     def comer(self):
        print("¿pollito?")
```

```python
class Corgui(Perro):
    def __init__(self, nombre):
        super().__init__(nombre)

    def comer(self):
        print(
            “Quiero mi pollito con”
            ”queso ahora!"
        )
```

```python
class Dachshund(Perro):
    def __init__(self, nombre):
        super().__init__(nombre)

    def comer(self):
        print("Me vas a dar pollito?")

    def comer(self, comida: Comida):
        if comida.tipo == “pollito”:
            print("Gracias por el pollito!")

    def comer(self, remedio: Remedio):
        print("Alejate de mi satanás!)
```

Multiherencia

Multiherencia

- ¿Cuál es la diferencia entre herencia simple y multiherencia?
- ¿Cuáles son sus ventajas?
- ¿Qué es el problema del diamante?
- ¿Qué significa MRO y como afecta en la herencia?

Ejemplo de multiherencia y MRO

```python
class Salchicha:
    def __init__(self):
        super.__init__()
        self.tipo = "Embutido"
        self.forma = "Alargado"
    def saludar(self):
        print("Soy un embutido")
```

```python
class Perro:
    def __init__(self):
        super.__init__()
        self.tipo = "Animal"
        self.sonido = "Guau"
    def saludar(self):
        print(f"{self.sonido}")
```

```python
class Demonio:
    def __init__(self):
        super.__init__()
        self.tipo = "Bestia"
        self.origen = "Infierno"
    def saludar(self):
        print("Te asustaré...")
```

```python
class Dachshund(Salchicha, Perro, Demonio):
     def __init__(self, nombre):
          super().__init__()
          self.nombre = nombre

     def saludar(self):
          print(f"Hola, mi nombre es {self.nombre} y soy un
     {self.tipo} de forma {self.forma} y vengo del
     {self.origen}")
          super().saludar()
```

Clases abstractas

Clases abstractas

- ¿Qué es una clase abstracta?
- ¿Cuándo es necesario usarla?
- ¿Cuál es el uso del módulo **abc**?

Decoradores

Decoradores

- ¿Qué son los objetos de primera clase?
- ¿Qué problema resuelven los decoradores?
- ¿Qué devuelve un decorador?
- ¿Qué diferencia hay entre acceder a un método normal y a una **`property`**?
- ¿Qué ventajas tiene usar **`@property`** frente a acceder directamente a atributos?

Decoradores

```python
import time
# Esta es la funcion decoradora
def agregar_tiempo(func):
    def dormir(n: int, m: int) -> int:
        time.sleep(2)
        print("Espere 2 segundos")
        return func(n, m)
    return dormir

# Aplicamos el decorador a sumar_algo
@agregar_tiempo
def sumar_algo(n: int, m: int) -> int:
    return n + m

print("Empieza")
print(sumar_algo(3, 2))
print("Termina")
```

Decoradores

sin decoradores

```python
class Planta:
    def __init__(self, nombre: str) -> None:
        self.nombre = nombre
        self.calidad = "bueno"

    def obtener_calidad(self) -> str:
        print("accediendo al getter")
        return "demasiado " + self.calidad

    def cambiar_calidad(self, nueva_calidad: str)
        self.calidad = nueva_calidad
        print("Parece que ahora soy un "
              f"{self.nombre} {self.calidad}.")

p = Planta("Zapallo")
print(p.calidad)
print(p.obtener_calidad())

p.cambiar_calidad("meh")

# ver el cambio
print(p.obtener_calidad())
```

con decoradores @propierties

```python
class Planta:
    def __init__(self, nombre: str) -> None:
        self.su_nombre = nombre
        self._calidad = "bueno"

    @property
    def calidad(self) -> str:
        print("accediendo al getter")
        return "demasiado " + self._calidad

    @calidad.setter
    def calidad(self, nueva_calidad: str) -> None:
        print("accediendo al setter")
        self._calidad = nueva_calidad
        print("Parece que ahora soy un "
              f"{self.su_nombre} {self._calidad}.")

p = Planta("Zapallo")
print(p._calidad)  # "bueno"

p.calidad = "regular"
print(p.calidad)
```

Veamos una pregunta de Evaluación Escrita

Tema: OOP (Midterm 2024-1)

7. En función del siguiente código, ¿cuál o cuáles clases se pueden instanciar sin provocar un error al momento de ejecutar el código?

   A) Solo la clase A
   B) Solo la clase B
   C) Solo la clase C
   D) Las clases B y C
   E) Ninguna se puede instanciar

```python
from abc import ABC, abstractmethod

class A(ABC):
    @abstractmethod
    def saludar(self):
        pass

    @abstractmethod
    def despedir(self):
        pass

class B(A):
    def saludar(self):
        print("Bonjour")

class C(B):
    def despedir(self):
        print("Au revoir")
```

Veamos una pregunta de Evaluación Escrita

Tema: OOP (Midterm 2024-1)

7. En función del siguiente código, ¿cuál o cuáles clases se pueden instanciar sin provocar un error al momento de ejecutar el código?

A) Solo la clase A
B) Solo la clase B
C) Solo la clase C
D) Las clases B y C
E) Ninguna se puede instanciar

```python
from abc import ABC, abstractmethod

class A(ABC):
    @abstractmethod
    def saludar(self):
        pass

    @abstractmethod
    def despedir(self):
        pass

class B(A):
    def saludar(self):
        print("Bonjour")

class C(B):
    def despedir(self):
        print("Au revoir")
```

Actividad

Comentarios AC3

Hagan `git add`, `git commit`, `git push`

- **Lean TODO con calma** antes de empezar a programar.

Programación Avanzada
IIC2233 2026-1

Jaime Middleton - Pablo Araneda - Antonio López - Tamara Vidal - Daniela Concha